

WAL Archiving — archive_command and WAL Management

Table of Contents

1. [Learning Objectives](#)
 2. [WAL Archiving Architecture](#)
 3. [Configuring archive_mode and archive_command](#)
 4. [WAL Segment Management](#)
 5. [Archive Storage Options](#)
 6. [pgBackRest WAL Archiving](#)
 7. [WAL-G for Cloud Storage](#)
 8. [Monitoring WAL Archiving](#)
 9. [WAL Archive Cleanup](#)
 10. [Troubleshooting](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Interview Questions](#)
 14. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Configure WAL archiving for PITR
 - Write and test archive_command scripts
 - Manage WAL segment lifecycle
 - Set up cloud-based WAL archiving with pgBackRest or WAL-G
 - Monitor and alert on archive failures
 - Clean up old WAL archives safely
-

WAL Archiving Architecture

WAL LIFECYCLE:

1. Transaction commits
|
▼
2. WAL record written to pg_wal/
(0000000100000000000001 - 16MB segment)
|
▼ (segment fills or archive_timeout fires)
3. archive_command triggered
Copies segment to archive storage
|
▼
4. Archive storage (NFS, S3, etc.)
0000000100000000000001

```
000000010000000000002
```

```
...
```

```
|
```

```
▼ (during recovery)
```

5. restore_command reads from archive
Applied to database during PITR

RETENTION:

Segments deleted from pg_wal when:

- They've been archived (archive_mode = on)
- They're beyond wal_keep_size
- No replication slots reference them

Configuring archive_mode and archive_command

```
# postgresql.conf

# Enable WAL archiving
archive_mode = on           # 'on' or 'always'
# 'on': Archive on primary only
# 'always': Archive on standby too (useful for cascading)

# Archive command (see examples below)
archive_command = 'command_here'

# WAL level
wal_level = replica

# Force WAL segment switch every N seconds (affects RPO)
archive_timeout = 300       # 5 minutes: RPO <= 5 min for idle DBs
# For tight RPO: 60 seconds (higher I/O cost)
# For loose RPO: 0 (wait for segment to fill)
```

archive_command Examples

```
# 1. SIMPLE LOCAL COPY (dev/test)
archive_command = 'test ! -f /mnt/archive/%f && cp %p /mnt/archive/%f'

# Explanation:
# test ! -f /mnt/archive/%f = only copy if file doesn't exist already
# cp %p /mnt/archive/%f     = copy from pg_wal to archive
# Idempotency check prevents overwriting if retry occurs

# 2. WITH TIMESTAMP SUBDIRECTORY
archive_command = 'mkdir -p /mnt/archive/$(date +%Y/%m/%d) && cp %p /mnt/archive/$(date +%Y/%m/%d)/%f'
# Organizes archives by date

# 3. COPY WITH COMPRESSION
```

```

archive_command = 'gzip < %p > /mnt/archive/%f.gz'
# restore_command: gunzip < /mnt/archive/%f.gz > %p

# 4. RSYNC TO REMOTE (with ssh key auth)
archive_command = 'rsync -a --ignore-existing %p backup@backup-
server:/archive/pg_wal/%f'

# 5. AWS S3
archive_command = 'aws s3 cp %p s3://my-pg-archive/wal/%f'
# Note: aws s3 cp is idempotent (overwrites if same name = harmless for WAL)

# 6. GCS (Google Cloud Storage)
archive_command = 'gsutil cp %p gs://my-pg-archive/wal/%f'

# 7. WITH ERROR HANDLING AND LOGGING
archive_command = '/usr/local/bin/archive_wal.sh %p %f'

```

```

#!/bin/bash
# /usr/local/bin/archive_wal.sh

WAL_PATH="$1"
WAL_FILE="$2"
ARCHIVE_DIR="/mnt/pg_archive"
LOG="/var/log/postgresql/archive.log"

echo "$(date) Archiving $WAL_FILE" >> "$LOG"

# Idempotency check
if [ -f "${ARCHIVE_DIR}/${WAL_FILE}" ]; then
    echo "$(date) $WAL_FILE already archived" >> "$LOG"
    exit 0
fi

# Copy with verification
cp "$WAL_PATH" "${ARCHIVE_DIR}/${WAL_FILE}.tmp"
if [ $? -ne 0 ]; then
    echo "$(date) ERROR: Failed to copy $WAL_FILE" >> "$LOG"
    rm -f "${ARCHIVE_DIR}/${WAL_FILE}.tmp"
    exit 1
fi

# Atomic rename (prevents partial files)
mv "${ARCHIVE_DIR}/${WAL_FILE}.tmp" "${ARCHIVE_DIR}/${WAL_FILE}"

echo "$(date) Successfully archived $WAL_FILE" >> "$LOG"
exit 0

```

Test archive_command

```

# Test manually before deploying
export PGDATA=/var/lib/postgresql/16/main

# Find a WAL file
WAL_FILE=$(ls ${PGDATA}/pg_wal/000000010000000000* 2>/dev/null | head -1)
WAL_FILENAME=$(basename "$WAL_FILE")

# Test the command
test ! -f /mnt/archive/${WAL_FILENAME} && cp ${WAL_FILE}
/mnt/archive/${WAL_FILENAME}
echo "Exit code: $?" # Must be 0

# Verify file was created
ls -la /mnt/archive/${WAL_FILENAME}

# Force archiving of current WAL
sudo -u postgres psql -c "SELECT pg_switch_wal();"
sleep 2

# Check pg_stat_archiver
sudo -u postgres psql -c "SELECT last_archived_wal, last_archived_time, failed_count
FROM pg_stat_archiver;"

```

WAL Segment Management

WAL Segment Size

```

# Default WAL segment size: 16MB
# Can be changed at initdb time:
initdb --wal-segsize=64 # 64MB segments (for high-throughput databases)
# Larger segments: fewer segments to archive, but coarser-grained PITR

# Check current segment size:
pg_controldata | grep "Bytes per WAL segment"
# Default: 16777216 (16MB)

```

WAL Retention in pg_wal

```

# How much WAL PostgreSQL keeps in pg_wal directory
wal_keep_size = 1GB # Keep at least 1GB of WAL in pg_wal
# This is a safety net for standbys without slots
# With archive_mode = on: WAL also stays until archived

# Replication slots prevent deletion:
# A slot with restart_lsn will prevent WAL deletion
SELECT slot_name, restart_lsn,

```

```
        pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS bytes_behind
FROM pg_replication_slots;
```

Manual WAL Switch

```
-- Force WAL segment rotation (useful to trigger archive_command test)
SELECT pg_switch_wal();
-- Returns the LSN where the old segment ended

-- Current WAL file name
SELECT pg_walfile_name(pg_current_wal_insert_lsn());

-- Current WAL position
SELECT pg_current_wal_lsn();

-- WAL directory size
SELECT pg_size_pretty(
    SUM(size)
) AS wal_dir_size
FROM pg_ls_waldir();
```

Archive Storage Options

Local Filesystem

```
# Dedicated disk for archive
# Mount: /mnt/pg_archive
# Ensure: postgres user has write permission
# Monitoring: disk usage alert at 80%

mkdir -p /mnt/pg_archive
chown postgres:postgres /mnt/pg_archive
chmod 750 /mnt/pg_archive

archive_command = 'test ! -f /mnt/pg_archive/%f && cp %p /mnt/pg_archive/%f'
restore_command = 'cp /mnt/pg_archive/%f %p'
```

NFS

```
# NFS mount for shared WAL archive
# /etc/fstab:
# nfs-server:/pg-archive /mnt/pg_archive nfs defaults,_netdev 0 0

mount -t nfs nfs-server:/pg-archive /mnt/pg_archive

archive_command = 'cp %p /mnt/pg_archive/%f'
restore_command = 'cp /mnt/pg_archive/%f %p'
```

```
# NFS considerations:
# - Ensure NFS is mounted before PostgreSQL starts
# - Monitor NFS connectivity
# - Consider rsync instead of cp for reliability:
archive_command = 'rsync --ignore-existing %p /mnt/pg_archive/%f && test -f
/mnt/pg_archive/%f'
```

Amazon S3

```
# Install aws-cli
sudo apt-get install -y awscli

# Configure IAM role or credentials
# Recommended: EC2 instance role (no credentials in config)

archive_command = 'aws s3 cp %p s3://my-bucket/pg-archive/%f 2>>
/var/log/pg_archive.log'
restore_command = 'aws s3 cp s3://my-bucket/pg-archive/%f %p 2>>
/var/log/pg_restore.log'

# With lifecycle policy: automatically move old WAL to Glacier after 30 days
# (via S3 bucket lifecycle configuration)
```

pgBackRest WAL Archiving

pgBackRest is the preferred tool for production WAL archiving. See also [06_pgbackrest.md](#).

```
# Install pgBackRest
sudo apt-get install -y pgbackrest

# Configure /etc/pgbackrest/pgbackrest.conf
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2
repo1-retention-archive=14    # Keep 14 days of WAL archives
process-max=4
log-level-console=info
log-level-file=detail
start-fast=y

[mydb]
pg1-path=/var/lib/postgresql/16/main
```

```
# postgresql.conf
archive_command = 'pgbackrest --stanza=mydb archive-push %p'
restore_command = 'pgbackrest --stanza=mydb archive-get %f %p'
```

```
# Initialize stanza
pgbackrest --stanza=mydb stanza-create

# Verify
pgbackrest --stanza=mydb check

# Test archiving
sudo -u postgres psql -c "SELECT pg_switch_wal();"
pgbackrest --stanza=mydb info
```

WAL-G for Cloud Storage

WAL-G is a cloud-native WAL archiving tool (S3, GCS, Azure, etc.).

```
# Install WAL-G
wget https://github.com/wal-g/wal-g/releases/download/v3.0.0/wal-g-pg-ubuntu-20.04-
amd64 \
    -O /usr/local/bin/wal-g
chmod +x /usr/local/bin/wal-g
```

```
# /etc/wal-g.conf
WALG_S3_PREFIX=s3://my-wal-archive/production
AWS_REGION=us-east-1
# AWS credentials via instance role (no hardcoded keys)
PGHOST=/var/run/postgresql
PGUSER=postgres
PGDATABASE=postgres
WALG_COMPRESSION_METHOD=lz4      # Fast compression
WALG_DELTA_MAX_STEPS=6           # Incremental backups
```

```
# postgresql.conf
archive_command = 'wal-g wal-push %p'
restore_command = 'wal-g wal-fetch %f %p'
```

```
# Take base backup
wal-g backup-push /var/lib/postgresql/16/main

# List backups
wal-g backup-list

# Delete old backups (keep last 7 full backups)
wal-g delete retain FULL 7
```

Monitoring WAL Archiving

```

-- Real-time archiving status
SELECT
    last_archived_wal,
    last_archived_time,
    last_failed_wal,
    last_failed_time,
    archived_count,
    failed_count,
    CASE
        WHEN failed_count = 0 THEN 'Healthy'
        WHEN last_failed_time > now() - INTERVAL '5 minutes' THEN 'FAILING'
        ELSE 'Previously failed'
    END AS status,
    now() - last_archived_time AS archive_lag
FROM pg_stat_archiver;

-- Alert: archiving failed in last 10 minutes
SELECT failed_count, last_failed_wal, last_failed_time
FROM pg_stat_archiver
WHERE last_failed_time > now() - INTERVAL '10 minutes';

-- Alert: archive is more than 10 minutes behind
SELECT now() - last_archived_time AS lag
FROM pg_stat_archiver
WHERE last_archived_time < now() - INTERVAL '10 minutes';

-- WAL generation rate
SELECT
    pg_size_pretty(
        pg_wal_lsn_diff(pg_current_wal_lsn(), '0/0') /
        EXTRACT(EPOCH FROM (now() - pg_postmaster_start_time()))
    ) AS wal_per_second;

```

Monitoring Shell Script

```

#!/bin/bash
# check_wal_archiving.sh

FAILED=$(psql -t -c "SELECT failed_count FROM pg_stat_archiver;")
LAST_FAILED=$(psql -t -c "SELECT last_failed_time FROM pg_stat_archiver;")
LAST_ARCHIVED=$(psql -t -c "SELECT last_archived_time FROM pg_stat_archiver;")

# Check for recent failures
if [ "$FAILED" -gt 0 ]; then
    echo "WARNING: $FAILED archive failures. Last: $LAST_FAILED"
    exit 1
fi

# Check archive is recent
AGE=$(psql -t -c "SELECT EXTRACT(EPOCH FROM (now() - last_archived_time)) FROM

```



```
pg_stat_archiver;")
if (( $(echo "$AGE > 600" | bc -l) )); then
    echo "WARNING: Last archive was $AGE seconds ago (> 10 minutes)"
    exit 1
fi

echo "OK: Archiving healthy. Last archived: $LAST_ARCHIVED"
exit 0
```

WAL Archive Cleanup

```
# DANGER: Never delete WAL archives manually without knowing your recovery window

# Safe cleanup: Use pgBackRest retention management
pgbackrest --stanza=mydb expire

# pgBackRest handles retention automatically based on:
# repo1-retention-full = 2    # Keep 2 full backups
# repo1-retention-archive = 14 # Keep 14 days of WAL

# Manual cleanup (only if you know what you're doing):
# Find oldest backup and its WAL requirement:
ls -la /backup/base/oldest_backup/backup_label
# The "CHECKPOINT LOCATION" shows the minimum WAL needed

# Using pg_archivecleanup
pg_archivecleanup /mnt/pg_archive oldest_required_wal_file

# Example:
pg_archivecleanup /mnt/pg_archive 0000000100000000000005
# Deletes all WAL older than 0000000100000000000005

# WARNING: If you delete WAL that a standby needs, standby must re-sync
```

Troubleshooting

```
# PROBLEM: archive_command failing
# Diagnose:
psql -c "SELECT last_failed_wal, last_failed_time FROM pg_stat_archiver;"
# Check PostgreSQL log:
grep "archive_command" /var/log/postgresql/postgresql-*.log | tail -20

# PROBLEM: archive_command not running at all
# Check:
psql -c "SHOW archive_mode;"    # Must be 'on'
psql -c "SHOW archive_command;" # Must not be empty
# Try manual WAL switch:
psql -c "SELECT pg_switch_wal();"
```

```
sleep 2
psql -c "SELECT * FROM pg_stat_archiver;"

# PROBLEM: Archive directory filling up
# Check why:
du -sh /mnt/pg_archive/
ls -lt /mnt/pg_archive/ | head # Most recent
ls -lt /mnt/pg_archive/ | tail # Oldest

# Find oldest backup that needs WAL
# Then run pg_archivecleanup to remove older WAL

# PROBLEM: Archiving slow (archiving lag)
# Check WAL generation rate vs archive rate
watch 'psql -c "SELECT now() - last_archived_time AS lag FROM pg_stat_archiver;"'
# Solutions: Faster storage, compress WAL before archiving, S3 multipart upload
```

Common Mistakes

1. **archive_command that hides errors** — `command || true` always returns 0
 2. **No test ! -f** in copy command — race conditions on retry
 3. **Wrong permissions on archive directory** — postgres user can't write
 4. **NFS not mounted at startup** — archive fails silently
 5. **Not monitoring failed_count** — archive has been broken for days
 6. **archive_timeout = 0** for idle databases — WAL never archived until full
 7. **Deleting archives without knowing oldest needed WAL** — can't recover from backup
 8. **Archive on same disk as PostgreSQL** — disk failure takes both
 9. **No S3 bucket versioning** — accidental overwrite of archive
 10. **Testing restore_command** but not archive_command — archive may work, restore may fail
-

Best Practices

1. **Monitor pg_stat_archiver continuously** — alert on any failure
 2. **Store archive on different storage** than PostgreSQL data
 3. **Enable S3 bucket versioning** for archive buckets
 4. **Set archive_timeout** appropriate to your RPO (300s for 5 min RPO)
 5. **Use pgBackRest or WAL-G** for production — they handle compression, parallelism, retention
 6. **Test restore_command** monthly by actually recovering from archive
 7. **Idempotency check** in archive_command — prevents issues on retry
 8. **Use atomic writes** (copy to .tmp, then rename) to prevent partial files
 9. **Implement retention policy** — don't let archives grow unbounded
 10. **Document recovery procedure** using the archive
-

Interview Questions

Q1: What is the difference between `archive_mode = on` and `archive_mode = always` ?

A: `archive_mode = on` only archives WAL on the primary. `archive_mode = always` archives on both primary and standby. Use `always` when you want the standby to also contribute to the WAL archive (e.g., for

cascading archiving, DR regions, or as a backup source when the primary is unavailable). Both modes require a PostgreSQL restart to change.

Q2: What happens if `archive_command` fails?

A: PostgreSQL retries the command. The WAL segment stays in `pg_wal/` and is not deleted until archiving succeeds. `pg_stat_archiver.failed_count` increments and `last_failed_wal / last_failed_time` are updated. PostgreSQL also logs an error. If archiving keeps failing, `pg_wal/` can fill up (if archiving is behind `wal_keep_size` + active replication slots). Monitor `failed_count` and alert immediately.

Q3: Why is `test ! -f archive_dir/%f && cp %p archive_dir/%f` better than just `cp %p archive_dir/%f` ?

A: PostgreSQL may call `archive_command` multiple times for the same segment (on retry after failure, or in crash recovery scenarios). If the file already exists in the archive, a simple `cp` would overwrite it — which could corrupt the archive if the retry happens with different file content. The `test ! -f` check makes it idempotent: only copy if not already present.

Q4: What is `archive_timeout` and how does it affect RPO?

A: WAL segments are 16MB. A busy database fills segments frequently. An idle database might not fill a segment for hours. `archive_timeout` forces a WAL segment switch after N seconds, ensuring the current segment gets archived even if not full. With `archive_timeout = 300`, the maximum time between archives is 5 minutes → $RPO \leq 5 \text{ minutes}$. Without it, an idle database's RPO could be hours.

Q5: How do you safely clean up old WAL archives?

A: Identify the oldest base backup you need to recover from. The `backup_label` in that backup shows its "START WAL LOCATION." WAL segments older than that start location are no longer needed. Use `pg_archivecleanup /archive_dir/<oldest_needed_wal_file>` to remove WAL older than the specified file. NEVER delete WAL that any standby or backup still references.

Q6: What is the `restore_command` and when is it used?

A: `restore_command` is the inverse of `archive_command`. During PITR recovery, when PostgreSQL needs a WAL segment that's not in `pg_wal/`, it calls `restore_command` to fetch it from the archive. Format: `restore_command = 'cp /archive/%f %p'`. PostgreSQL substitutes `%f` with the filename and `%p` with the destination path. It must exit 0 on success and non-zero if the file doesn't exist (so PostgreSQL knows to try the next segment).

Q7: Why use `pgBackRest` for WAL archiving instead of a simple `cp` command?

A: `pgBackRest` adds: (1) Parallel WAL archiving (higher throughput). (2) Automatic compression. (3) Checksums for archive integrity. (4) Built-in retention management. (5) Native S3/GCS/Azure support. (6) Integration with `pg_basebackup` for coordinated backup + archive. (7) Archive verification commands. A simple `cp` command is fine for small/test environments but lacks these production features.

Exercises and Solutions

Exercise 1: Write and Test a Robust `archive_command`

```
#!/bin/bash
# /usr/local/bin/pg_archive_wal.sh
```

```

set -euo pipefail

WAL_FILE_PATH="$1"      # %p - full path
WAL_FILE_NAME="$2"      # %f - filename

ARCHIVE_DIR="/mnt/pg_archive"
S3_BUCKET="s3://my-pg-wal-archive"
LOG="/var/log/postgresql/wal_archive.log"

log() { echo "$(date '+%Y-%m-%d %H:%M:%S') $1" >> "$LOG"; }

# First: archive locally
if [ ! -f "${ARCHIVE_DIR}/${WAL_FILE_NAME}" ]; then
    cp "${WAL_FILE_PATH}" "${ARCHIVE_DIR}/${WAL_FILE_NAME}.tmp"
    mv "${ARCHIVE_DIR}/${WAL_FILE_NAME}.tmp" "${ARCHIVE_DIR}/${WAL_FILE_NAME}"
    log "Local archive: ${WAL_FILE_NAME}"
fi

# Then: sync to S3 (non-blocking for local archive success)
aws s3 cp "${ARCHIVE_DIR}/${WAL_FILE_NAME}" "${S3_BUCKET}/${WAL_FILE_NAME}" \
    2>> "$LOG" && log "S3 archive: ${WAL_FILE_NAME}" || \
    log "WARNING: S3 archive failed for ${WAL_FILE_NAME} (local archive OK)"

exit 0

# Test this script:
# /usr/local/bin/pg_archive_wal.sh
# /var/lib/postgresql/16/main/pg_wal/0000000100000000000001 0000000100000000000001
# echo "Exit: $?"

```

Cross-References

- [04_pitr.md](#) — Using WAL archive for PITR
- [06_pgbackrest.md](#) — pgBackRest WAL integration
- [03_pg_basebackup.md](#) — Base backup paired with WAL archiving
- [07_disaster_recovery.md](#) — WAL in DR scenarios